DeLP3E Approximation Experiment

Feature Reference

Definitions and computation methods
for every column in `results/all_results.csv`.

11 sections · 50 features

Generated by sampling/generate_feature_docs.py

Identifiers

config
string · categorical (e.g. sss, sms, ssc, mss, ccs)

Configuration code XYZ encoding the complexity level of the AM, AF and EM components (s = small, m = medium, c = complex).

How computed: Read directly from the directory name of the model repository.

model
string · model instance name (e.g. 0model, 14model, 87model)

Identifier of the specific model instance within a configuration.

How computed: Read from the model's JSON filename, stripping the .json extension.

literal
string · DeLP literal (e.g. a_7, ~d_13)

The queried literal whose posterior belief interval is being approximated. May be negated (prefixed with ~).

How computed: Selected from the model's exact file: only literals flagged as INTEREST (exact width <= 0.5) are queried.

Model structural features (EM / AF / AM sizes)

em_var
int · number of BN random variables

Size of the Epistemic Model. Determines the world space cardinality n_worlds = 2^em_var.
How computed: Read from the model JSON's 'em_var' field.

n_annots
int · count of non-trivially annotated AF rules

Number of AF rules that have a real annotation formula (not empty, not 'True', not 'not True'). Determines the program space cardinality n_programs = 2^n_annots.
How computed: Loop over the model's 'af' list and count entries whose annotation string is non-trivial.

n_rules
int · total AF rules

Total number of rules in the AF, including unannotated ones.
How computed: Length of the model's 'af' list.

annots_minus_em
int · = n_annots - em_var

Key theoretical predictor for choosing the sampling approach. Positive => program space is larger than world space => World Sampling preferred. Negative => Program Sampling preferred.
How computed: Computed as the difference of the two columns above.

n_worlds
int · = 2^em_var

Total number of possible BN worlds (full joint state space).
How computed: 2 raised to the power of em_var.

n_programs
int · = 2^n_annots

Total number of possible DeLP programs derivable from the AF (each annotated rule can be present or absent).
How computed: 2 raised to the power of n_annots.

Exact ground truth

exact_l
float · in [0, 1]
Exact lower bound of the queried literal's belief interval, computed by exhaustive enumeration of worlds.
How computed: Read from the literal's entry in {N}model_e_w.json.

exact_u
float · in [0, 1]
Exact upper bound of the belief interval.
How computed: Read from the same exact file as exact_l.

exact_width
float · = exact_u - exact_l
Width of the exact interval. 0 = the probability is known precisely; values close to 0.5 indicate maximal uncertainty.
How computed: Computed from the two columns above.

World Sampling results

worlds_l
float · in [0, 1]

Lower bound of the belief interval approximated by World Sampling within the configured time budget.

How computed: Output of the World Sampling algorithm in sampling/byWorldSampling.py.

worlds_u
float · in [0, 1]

Upper bound of the World Sampling approximation.

How computed: Output of byWorldSampling.py.

worlds_quality
float · = (1 - (worlds_u - worlds_l)) / (1 - exact_width)

Quality of the World Sampling approximation. 1.0 means the approximate interval matches the exact width; values below 1.0 indicate a wider, more conservative approximation.

How computed: Computed in run_experiment.py from the bounds and the exact width.

worlds_solver_time
float · seconds

Total wall-clock time spent by World Sampling for this model (across all queried literals).

How computed: Reported by byWorldSampling.py.

worlds_delp_calls
int · count

Number of DeLP solver invocations performed during World Sampling.

How computed: Reported by byWorldSampling.py.

Program Sampling results

progs_l
float · in [0, 1]
Lower bound of the belief interval approximated by Program Sampling within the configured time budget.
How computed: Output of sampling/byProgramSampling.py.

progs_u
float · in [0, 1]
Upper bound of the Program Sampling approximation.
How computed: Output of byProgramSampling.py.

progs_quality
float
Quality of the Program Sampling approximation. Same formula as worlds_quality.
How computed: Computed in run_experiment.py.

progs_solver_time
float · seconds
Wall-clock time spent by Program Sampling.
How computed: Reported by byProgramSampling.py.

progs_delp_calls
int · count
Number of DeLP solver invocations during Program Sampling.
How computed: Reported by byProgramSampling.py.

Outcome (target variable)

winner
string in {worlds, programs, tie}
Which sampling approach achieved higher quality within the time budget. The target for the master decision tree.
How computed: Comparison of worlds_quality vs progs_quality. 'tie' if the difference is exactly zero (very rare).

Literal-level features (computed from the AF)

lit_is_negated
int · {0, 1}

1 if the queried literal starts with the negation symbol '~', 0 otherwise.
How computed: literal.startswith('~').

lit_head_def
int · count

Number of defeasible rules (using the '-<' connector) where the queried literal appears as the head.
How computed: Parse every rule of the AF: count those where the head matches the queried literal AND the rule type is defeasible.

lit_head_strict
int · count

Number of strict rules (using the '<-' connector) where the queried literal appears as the head.
How computed: Same as lit_head_def but for strict rules.

lit_body_count
int · count

Number of rules where the queried literal appears in the body (after the '-<' or '<-' connector).
How computed: Tokenize the rule body by commas and check for an exact match.

lit_is_fact
int · {0, 1}

1 if the literal is a non-annotated fact (a rule of the form 'lit -< true.' with no annotation formula); 0 otherwise.
How computed: Detected by checking that the rule's body equals 'true' AND the annotation column is trivial.

lit_is_ann_fact
int · {0, 1}

1 if the literal is an annotated fact (rule body is 'true' AND the annotation formula is non-trivial); 0 otherwise.
How computed: Same body check as lit_is_fact, but with a non-trivial annotation.

lit_complement_body
int · count

Number of rules in the AF where the COMPLEMENT of the queried literal (e.g. ~a_7 if literal is a_7) appears in the body.
How computed: Compute the complement (toggle '~' prefix) and count body matches.

lit_complement_head
int · count

Number of rules where the complement of the queried literal appears as the head — i.e. rules supporting the opposite conclusion.
How computed: Compute the complement and count head matches.

AF global features (basic)

af_pct_annotated
float · in [0, 1]

Fraction of AF rules that carry a non-trivial annotation formula. Measures how densely the program is coupled to the BN.
How computed: n_annots / n_rules.

af_avg_annot_vars
float

Average number of BN variable indices referenced per non-trivial annotation formula. Higher values mean richer coupling.
How computed: Use a regex \d+ on each non-trivial annotation, take the count, average across annotations.

af_avg_connectors
float

Average number of logical connectors (and / or / not) per non-trivial annotation formula.
How computed: Regex \b(and|or|not)\b on each annotation, average the counts.

af_avg_body_size
float

Average number of literals in the body of non-fact rules.
How computed: For each rule whose body is not 'true', count the comma-separated literals, then average.

af_max_body_size
int · count

Maximum body size across all non-fact rules. Indicates the most complex dependency chain in the program.
How computed: Maximum of the per-rule body sizes.

AM complexity features (from the dialectical graph)

am_n_arguments
int · count

Total number of arguments built by the DeLP solver when querying all literals of the program.
How computed: Run the solver in 'all' mode and count entries in the dGraph field whose argument key contains a comma (i.e. multi-rule arguments, excluding trivial single-fact arguments).

am_n_defeaters
int · count

Total number of defeat relations across all arguments.
How computed: Sum the lengths of the 'defeats' lists for every counted argument in the dGraph.

am_n_trees
int · count

Number of dialectical trees evaluated by the solver. Each tree explores one root argument's defeat structure.
How computed: Walk the 'status' field of the solver output: count root entries whose argument string contains '-<' AND have at least one child in the lines (defeat relations).

am_avg_def_rules
float · ALE in DeLP3E nomenclature

Average number of defeasible rules per argument. Higher values mean longer derivation chains.
How computed: n_def_rules / n_arguments, where n_def_rules counts subarguments minus trivial presumptions for each argument.

am_avg_arg_lines
float · AWT

Average number of argument lines per dialectical tree. A measure of branching factor.
How computed: n_arg_lines / am_n_trees.

am_avg_height_lines
float · AHE

Average height of argument lines across all trees. A measure of the depth of dialectical reasoning.
How computed: Sum of recursive line heights divided by total argument lines.

EM complexity features (Bayesian Network)

em_n_arcs
int · count

Number of directed arcs in the BN.
How computed: Length of bn.arcs() after loading the .bifxml file with pyAgrum.

em_treewidth
int

Treewidth of the BN's moralized undirected graph. Indicates the intrinsic complexity of exact inference: lower is easier.
How computed: Build the moral graph (add edges between co-parents), then call networkx.algorithms.approximation.treewidth_min_degree.

em_avg_in_degree
float

Average number of parents per BN node.
How computed: Mean of |parents(n)| across all nodes.

em_max_in_degree
int

Maximum number of parents that any single BN node has.
How computed: Max of |parents(n)| across all nodes.

em_entropy
float · bits

Joint entropy H(X1, ..., Xn) of the BN. Measures how much uncertainty / spread the BN has across worlds.
How computed: If 2^em_var <= 262144, exact enumeration: H = -sum p log2 p over all worlds. Otherwise Monte Carlo: draw samples, average -log2 p.

AF complexity features (extended)

af_n_em_vars_used

int · count

Number of distinct BN variables that appear at least once across all annotation formulas in the AF.

How computed: Collect every \d+ token from every non-trivial annotation into a set and report its size.

af_avg_complexity

float

Average complexity per non-trivial annotation, where complexity = number of variables + number of connectors.

How computed: For each annotation, count vars + connectors; average over non-trivial annotations.

af_max_complexity

int · count

Maximum (vars + connectors) across all annotations. The single most complex annotation in the program.

How computed: Maximum of the per-annotation complexity.